Garbage Classification Model

ENSF 617 – Assignment 2

Contributors:

Conner Castle

Saad Ahmad Malik

Toshi Biswas

William Watson

Introduction

This document is intended to accompany the deep learning model designed to classify images of garbage into the appropriate disposal category: black (landfill), green (organics), or blue TTR (recycling). The task is formulated as a supervised multiclass classification problem using the CVPR 2024 dataset, where each sample consists of an image and associated filename text. The goal is to learn a model that can accurately predict the correct bin from these inputs.

To accomplish this, we implement a multimodal architecture that combines visual and textual features. A pretrained image model is used to extract image embeddings, while a text model encodes text derived from the filenames. The two feature vectors are projected into a common space, normalized, concatenated, and passed through a fully connected classifier to produce the final prediction. The model is trained end-to-end with partial fine-tuning of the pretrained backbones.

Dataset

The model was trained and evaluated using the provided garbage classification dataset. The dataset is organized into training, test, and validation. Each set contains images grouped into class-specific folders corresponding to the target disposal categories (black, green, and blue TTR). The file names contain short descriptive text that is used as an additional text-based signal alongside the image data.

The filename text is processed to extract the words. Images are resized to 224×224 pixels and normalized, with random horizontal flipping applied during training for data augmentation. The processed text and images are paired and loaded through a custom PyTorch Dataset to support multimodal training.

Methodology

We implemented a multimodal classification architecture that combines image and text features through a concatenation design. In this design, we take the embeddings from two models, and combine them into a single input vector that becomes input to a fully connected neural network with four output classes. For the image model, both MobileNetV2 and EfficientNetV2 were evaluated as the pre-trained model to produce image embeddings. MobileNetV2 was chosen over larger architectures such as ResNet because it is designed to be lightweight and computationally efficient, making it suitable for deployment on devices with limited resources. Since our multimodal architecture already includes DistilBERT for text processing, using a smaller image backbone helps keep the overall model size and training cost manageable. Additionally, MobileNetV2 maintains strong performance while using significantly fewer parameters than ResNet, aligning well with the efficient design philosophy of DistilBERT. Most of the convolutional layers are frozen to preserve pretrained features. The text branch uses DistilBERT to encode the text descriptions and produce additional embeddings. Each set of embeddings is passed through a linear layer that outputs a fixed size vector and then each is normalized before concatenation. These steps are to prevent one model dominating the output. The normalized features are then concatenated and passed through a fully connected network with dropout for regularization.

Training Setup

The model training parameters were varied in several experiments to empirically optimize the accuracy. Several parameters were varied, including:

- Models – Tested MobileNet and EfficientNet for images and DistilBert for text
- Number of epochs – Ranged from 4 - 20
- Batch size – Ranged from 8 - 32
- Image size – Tested both 224x224 and 480x480

After each epoch, the model is evaluated on the validation set. The version of the model with the lowest validation loss is saved as the best model. Once training is finished, this best model is used to make predictions on the test set, and performance is reported using overall accuracy and a confusion matrix.

Subset of Experiments Run

Number	Image Model	Text Model	Fusion Strategy	Config	Results
1	EffNet V2	None	None	Hardware: M1 MBP LR: 1e-3 Epochs: 10 Batch Size: 32 Image Size: 224x224 Num Workers: 4	Train time: 24 min Best Epoch: 07 Train Loss: 0.7692 Val Loss: 0.7236 Val Accuracy: 72.06%
2	EffNet V2	None	None	Hardware: M1 MBP LR: 1e-3 Epochs: 10 Batch Size: 24 Image Size: 480x480 Num Workers: 4	Train time: 96 min First Epoch: 77% Best Epoch: 07 Train Loss: 0.7692 Val Loss: 0.7236 Val Accuracy: 79.3% Test Accuracy:73.3%
3	EffNet V2	DistilBERT	Late Fusion	Hardware: M1 MBP LR: 1e-3 Epochs: 10 Batch Size: 24 Image Size: 224x224 Num Workers: 4	Train time: 32 min First Epoch: 83.8% Best Epoch: 07 Train Loss: 0.47 Val Loss: 0.35 Val Accuracy: 87.0% Test Accuracy: 82.0%
4	MobileNetV2	DistilBERT	Late Fusion, Unfroze last 2 layers of each pre-trained	Hardware: RTX 4060 Head LR: 2e-3 Image Model LR: 1e-4 Text Model LR: 2e-5 Epochs: 10 Batch Size: 32 Image Size: 224x224 Num Workers: 0	Train time: 70min Train Loss: 0.19698 Val Loss: 0.30361 Test Accuracy: 84.5%

Final Model Training Results

The loss curves indicate the model may be overtraining after epoch 4. This could be reduced through regularization techniques, implementing a better dropout routine, or adding more data and data augmentation to the samples.

Discussion and Conclusions

Best Model

The best model that was tested was experiment 4, using a late fusion model with unfreezing the last two layers of each pre-trained backbone. This model achieved 84.7% Test Accuracy. This model was trained for 4 epochs.

Learning Rates (2e-5, 1e-4, 2e-3)

We used differential learning rates when training the model. Since the pretrained backbones (mobilenet_v2 and DistilBERT) were fine-tuned rather than fully retrained, smaller learning rates were assigned to them to prevent large updates that could disrupt pretrained representations. Specifically, we used a learning rate of **2e-5** for the text encoder and **1e-4** for the image encoder.

In contrast, the projection and classifier (fully connected) layers were trained with a higher learning rate of **2e-3**, as these layers were trained from scratch and required faster adaptation to the task. This strategy allows stable fine-tuning of pretrained components while enabling efficient learning in newly initialized layers.

Batch size (32)

Smaller batch sizes introduce more gradient noise, which can sometimes improve generalization but may slow convergence. Larger batch sizes produce smoother gradient updates but require more memory. The chosen batch size provided stable training while remaining computationally feasible.

Number of epochs (10)

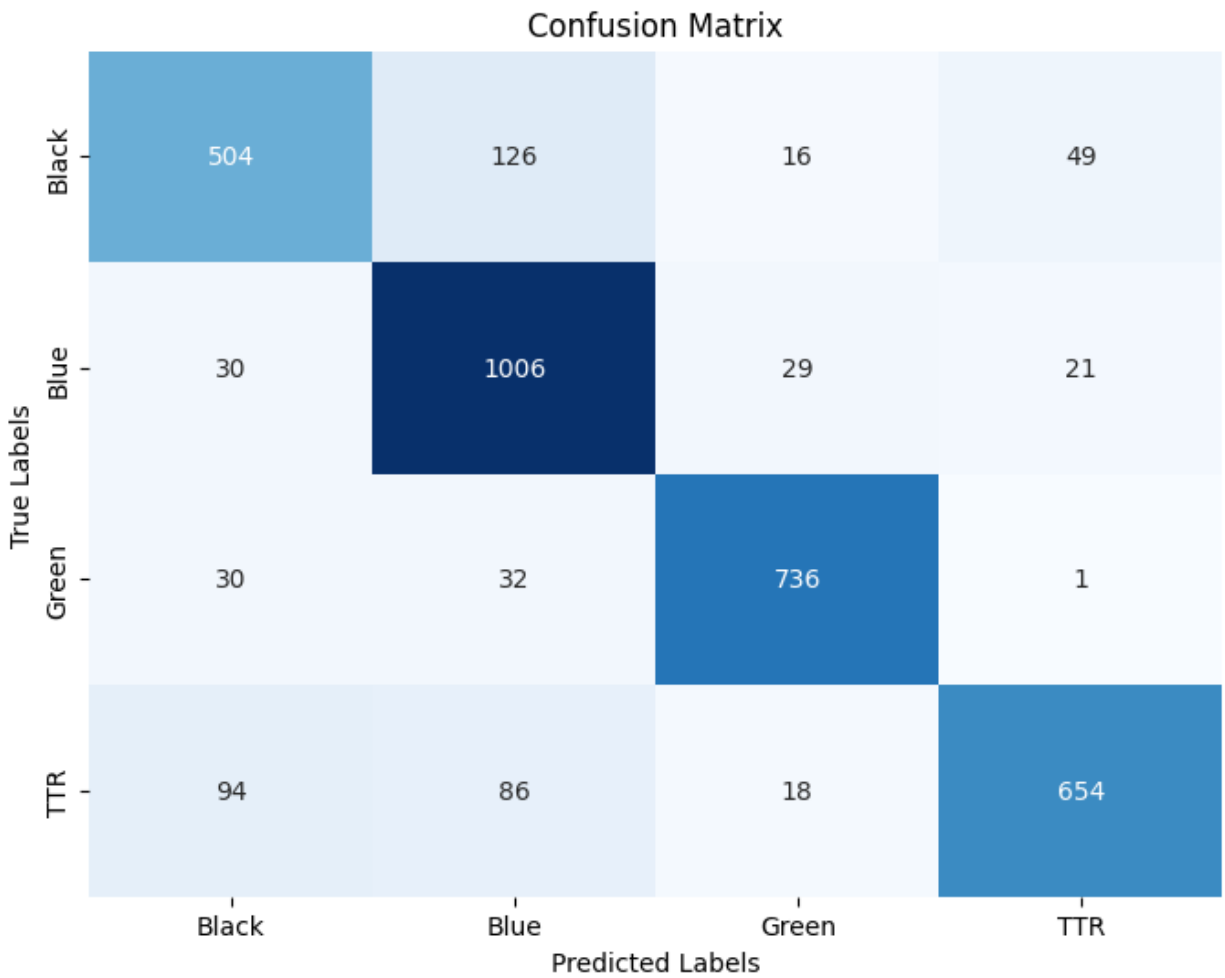
The model was trained for 10 epochs, which was sufficient for convergence given that only shallow layers were trained. Since the backbones were frozen, the model did not require extensive training to adapt.

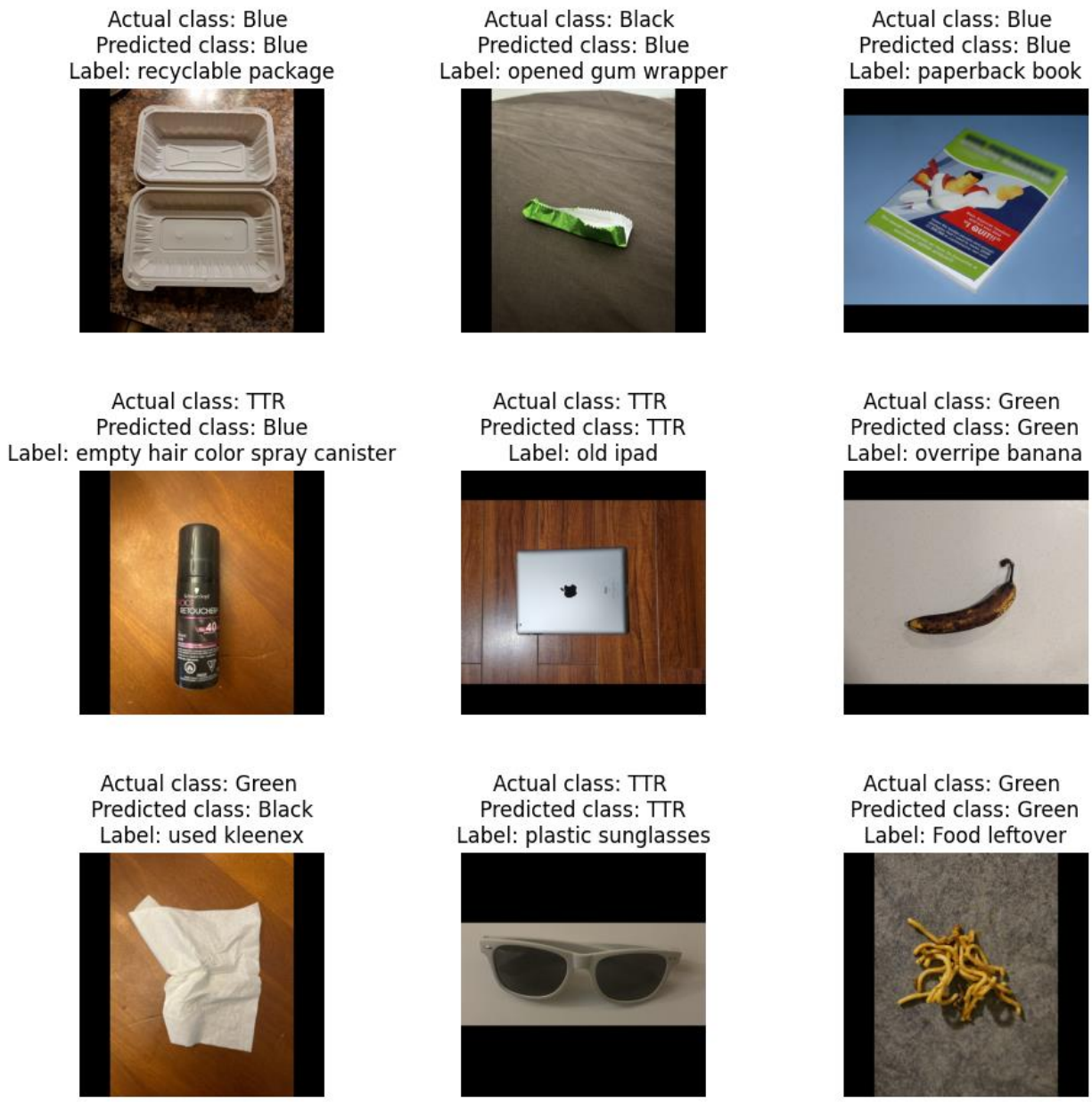
Optimizer (AdamW)

The AdamW optimizer was chosen due to its adaptive learning rate mechanism. Since only projection and classifier layers were trained, AdamW helps achieve fast and stable convergence without extensive hyperparameter tuning.

Confusion Matrix (best model)

Below is a confusion matrix showing the results of the best model. Note that the model has a bias towards the blue category, which is expected since it has the most data in the training set. However, it also has the most false positives, with many black and TTR images classified as blue.



Random Image Visualizer

Below is a sample output of the classifier in action, where nine random images from the test set are displayed along with the true label, predicted label, and the associated text description. This figure helps give an idea of the types of failure modes for the model. For instance, it would be reasonable to assume that a used Kleenex would go in the trash instead of composting, but a failure to recognize an empty hair spray can indicates an issue that would need resolved.